

Task-5

WRITING JOIN QUERIES, EQUIVALENT AND/OR) RECURSIVE QUERIES

Aim:

To implement and execute join queries, equivalent queries, and recursive queries using a university database scenario.

Procedure:

The SQL joins clause is used to combine records from two or more table in a database. A join is a means for combining fields from two tables by using values common to each. The join is actually performed by the 'where' clause which combines specified rows of tables.

* Create the database and tables (Students, Department, Courses, Environments).

* Write SQL Queries using different types of joins

* Insert sample data

* Display result and Verify Correctness.

Syntax:

```
SELECT Column 1, Column 2, Column 3, From table-name 1,  
table-name 2 WHERE table-name 1, Column = table-name 2,  
Column Name;
```

1) - types of joins:

1. Simple joins

2. Self join

3. Outer join

* Inner join:

Returns records that have matching values in both tables

```
SELECT Column-name (s) from table
```

```
INNER JOIN table 2 ON table 1. Column-name = table 2. Column-name,
```

* LEFT (Outer) JOIN

Return all records from the left table, and the matched records from the right table

```
SELECT Column-name (s) from table 1
```

```
LEFT JOIN table 2 ON table 1, Column-name = table 2. Column-name
```

(Left Outer) join

Return all records from the table and the matched records from the right table

```
SELECT m.phone_id, m.brand, m.model, s.ram  
s.storage, s.battery FROM mobiles m
```

LEFT JOIN Phone Specifications on m.phone_id : s.phone_id

Insert INTO Payment Values (30000, 15000, 25000, 2023-05-01)

1 Row (Completed)

Result: Records inserted successfully

JOIN QUERIES

Create table Customer (

Cust_ID INT PRIMARY KEY;

Cust_NAME VARCHAR (50) NOT NULL;

);

Create table mobile (

Mobile ID INT PRIMARY KEY;

Brand NARCHR (50) NOT NULL;

Mobile VARCHAR (50) NOT NULL;

Price Decimal (10, 2), CHECK (Price, 30,000);

);

CREATE TABLE Purchase (

Purchase ID INT PRIMARY KEY

Cust_ID NOT NULL;

Mobile_ID NOT NULL;

Quantity INT CHECK (Quantity > 0);

Purchase Date DATE DEFAULT CURRENT DATE;

FOREIGN KEY (Cust_ID);

REFERENCES Mobiles (Mobile ID)

);

CREATE TABLE Payment (

Payment ID INT PRIMARY KEY,

Purchase ID INT UNIQUE,

Amount Decimal (10, 2) NOT NULL;

Phone id	Brand	model	Price	ram	Storage	Battery
1	Realme	14D10	30000	16GB	256GB	3000mAh
2	Redmi	10050	15000	8GB	128GB	4500mAh
3	vivo	13pro	25000	12GB	256GB	5500mAh

Phone id	Brand	model	Price	ram	Storage	Battery
1	realme	14pro	3000	16GB	256GB	5000mAh
2	redmi	10pro	15000	8GB	128GB	4500mAh
3	vivo	13pro	25000	12GB	256GB	5500mAh

Full Outer Join

Phone id	Brand	model	Price	ram	Storage	Battery
1	realme	14 pro	30000	16GB	256GB	5000mAh
2	redmi	10pro	15000	8GB	128GB	2,500mAh
3	vivo	T3pro	25000	18GB	256GB	5500mAh

payment Data Deposit.

Current Date,

2. Right Outer Join:

```
SELECT m.phone_id, m.brand, m.model, s.ram, m.storage,
```

m.battery

FROM mobile phone s m

Full Outer JOIN Phone Specification

ON m.phone_id = s.phone_id,

3. Full Outer Join:

```
SELECT m.phone, m.brand, m.model, s.ram, s.storage -  
s.battery.
```

FROM mobile phones m.

Full Outer JOIN phone Specification ON m.phone_id =

s.phone_id,

4. Equivalent Queries.

```
SELECT s.mobile Name M Model Name
```

```
FROM mobile phone
```

```
JOIN Brand ON s.phone ID = M.phone ID. using Subquery
```

```
SELECT mobile Name;
```

```
(SELECT Brand Name from B v and B.
```

```
where M.phone ID = s.phone ID) AS
```

```
Model Name from mobile phone;
```

5. Recursive Query (purchase hierarchy).

with Recursive Purchases.

```
SELECT payment ID, phone ID
```

```
FROM Pre requests
```

```
UNION
```

```
SELECT payment ID, phone ID
```

```
FROM Pre requests P.
```

Result: the implementation of SQL Commands using joins & recursive queries are executed successfully.

VEL TECH - CSE	
EX NO.	
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	15
SIGN WITH DATE	16/9